

Modul 3: Blade Template Laravel

3.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Mengetahui konsep Blade Template di Laravel.
2. Memahami sintaks dasar Blade.
3. Mampu menggunakan direktif (directive) Blade untuk percabangan dan perulangan.
4. Mampu menggunakan fitur layouting dengan @extends, @section, dan @yield.
5. Menggunakan komponen Blade (@include dan komponen custom) untuk membuat tampilan dinamis.
6. Menerapkan Blade untuk men-generate konten web yang terstruktur dan mudah dikelola.

3.2 Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

Perangkat Keras:

- Komputer atau laptop dengan sistem operasi Windows/Linux/macOS.
- Minimal prosesor 1 GHz, RAM 4GB (disarankan 8GB), serta ruang penyimpanan kosong minimal 2GB.

Perangkat Lunak:

- XAMPP atau Laragon sebagai server lokal (Apache & MySQL).
- Composer sebagai dependency manager PHP.
- PHP versi 8.0 atau lebih tinggi (sudah termasuk dalam XAMPP/Laragon).
- Laravel Framework (minimal versi 8) yang sudah terinstal melalui Composer.
- Teks Editor / IDE seperti Visual Studio Code, Sublime Text, atau PHPStorm.
- Browser Web seperti Google Chrome, Safari, atau Mozilla Firefox untuk menjalankan aplikasi.

3.3 Tugas Pendahuluan

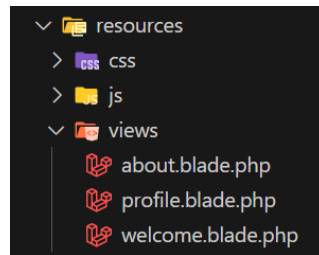
1. Apa itu Blade Template dalam Laravel?
2. Apa perbedaan antara Blade Template dengan file PHP/HTML biasa?
3. Sebutkan minimal 3 directive (perintah khusus) dalam Blade!
4. Jelaskan fungsi @extends dan @yield dalam Blade!
5. Apa keuntungan menggunakan @include pada Blade?

3.4 Materi Praktikum

3.4.1 Konsep Blade Template

Blade adalah template engine bawaan Laravel yang digunakan untuk menyusun tampilan (view). File Blade disimpan di folder resources/views dengan ekstensi .blade.php. Di dalam file Blade, kita dapat menulis HTML biasa yang dikombinasikan dengan sintaks khusus, seperti {{ }} untuk menampilkan data, serta directive seperti @if, @foreach, dan @extends untuk mengatur logika tampilan. Saat halaman diakses, Laravel mengompilasi file Blade menjadi kode PHP biasa dan menyimpannya di cache (storage/framework/views) sehingga tetap cepat dijalankan. Setelah

mengerjakan Tugas Pendahuluan, Anda tentu sudah memahami konsep Blade secara teori. Gambar di bawah ini menunjukkan struktur file view yang akan dibuat pada praktikum ini.



3.4.2 Sintaks Dasar Blade (Layout Utama)

Setelah memahami konsep Blade, selanjutnya kita akan mencoba mengimplementasikannya. Praktikum ini menggunakan pola MVC (Model-View-Controller), yaitu cara menyusun aplikasi dengan membagi kode menjadi beberapa bagian sesuai tugasnya masing-masing. Pembagian ini membuat kode lebih teratur, mudah dicari, dan mudah diperbaiki ketika terjadi kesalahan. Agar mudah dibayangkan, perhatikan analogi sebuah rumah makan: pelanggan memesan menu kepada pelayan (route), pelayan meneruskan pesanan ke dapur (controller) yang menyiapkan makanan (data), lalu makanan disajikan di atas piring (view) sehingga siap dinikmati pelanggan. Pada praktikum ini, peran masing-masing bagian adalah sebagai berikut:

- Route (routes/web.php) → menentukan alamat URL yang dapat diakses dan controller mana yang dijalankan ketika alamat tersebut dibuka.
- Controller (app/Http/Controllers/StudentController.php) → menyiapkan data mahasiswa, lalu mengirimkannya ke view.
- View (file .blade.php di folder resources/views) → menampilkan data dari controller dalam bentuk halaman HTML.
- Model → belum digunakan pada praktikum ini karena data masih ditulis langsung dalam bentuk array, belum diambil dari database.

Sintaks dasar Blade, seperti `{{ }}` untuk menampilkan data dan directive `@foreach` untuk perulangan, digunakan di dalam view untuk menampilkan data yang dikirim dari controller. Secara berurutan, alur kerja yang akan dibuat adalah sebagai berikut:

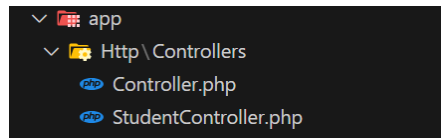
- Pengguna membuka alamat `http://127.0.0.1:8000/students` pada browser.
- Laravel mencocokkan URL `/students` dengan route di `routes/web.php`, lalu menjalankan method `index()` pada `StudentController`.
- Method `index()` menyiapkan data mahasiswa dan mengirimkannya ke view `students/index.blade.php`.
- View `students/index.blade.php` mewarisi kerangka halaman dari layout utama `layouts/main.blade.php`, lalu mengisi bagian judul dan konten dengan data mahasiswa.
- Laravel mengirimkan halaman HTML yang sudah jadi ke browser untuk ditampilkan kepada pengguna.

Dengan demikian, pada bagian ini Anda akan membuat tiga file baru, yaitu `StudentController.php`, `layouts/main.blade.php`, dan `students/index.blade.php`, serta menambahkan kode route pada file `routes/web.php`. Ikuti langkah-langkah di bawah ini secara berurutan dan pastikan setiap file disimpan sebelum melanjutkan ke langkah berikutnya.

1. Buat controller dengan perintah berikut:

```
php artisan make:controller StudentController
```

Perintah ini otomatis membuat file `app/Http/Controllers/StudentController.php` beserta kerangka class-nya. Jika berhasil, terminal menampilkan pesan `created successfully`.



2. Buka file `app/Http/Controllers/StudentController.php` yang baru dibuat, lalu ganti seluruh isinya dengan kode di bawah ini, lalu simpan (`Ctrl + S`).

```
StudentController.php
app > Http > Controllers > StudentController.php
1  <?php
2
3  namespace App\Http\Controllers;
4  use Illuminate\Http\Request;
5  class StudentController extends Controller
6  {
7      public function index()
8      {
9          $students = [
10             [
11                 'name' => 'Wahyudi',
12                 'major' => 'Informatika',
13                 'age' => 22,
14                 'courses' => ['Pemrograman Web', 'Database', 'Cloud Computing'],
15             ],
16             [
17                 'name' => 'Siti',
18                 'major' => 'Sistem Informasi',
19                 'age' => 21,
20                 'courses' => ['UI/UX Design', 'Manajemen Proyek', 'IoT'],
21             ],
22         ];
23
24         return view('students.index', compact('students'));
25     }
26 }
27
```

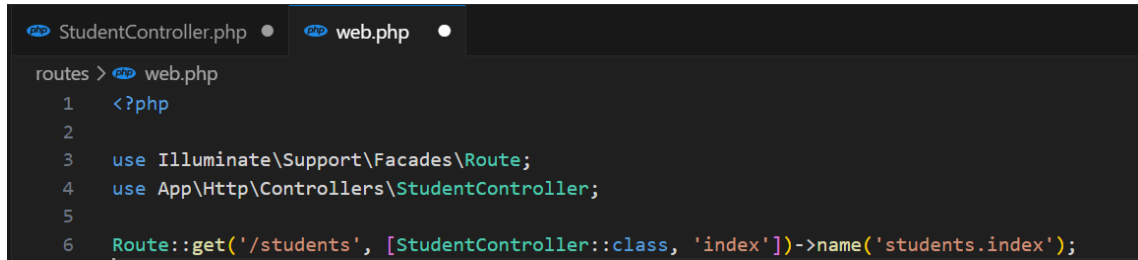
Penjelasan:

- `namespace App\Http\Controllers;` → menentukan lokasi class di dalam proyek Laravel.
- `class StudentController extends Controller` → mendefinisikan controller untuk mahasiswa.
- `public function index()` → method yang akan dijalankan ketika route dipanggil.
- `$students = [...]` → array multidimensi yang berisi data mahasiswa (nama, jurusan, umur, daftar mata kuliah).

- `return view('students.index', compact('students'));` → mengirim data `$students` ke view `resources/views/students/index.blade.php`.

Controller berperan sebagai penyiap data. Pada praktikum ini data masih berupa array biasa (belum menggunakan database), tetapi cara mengirimkannya ke view tetap sama. Fungsi `compact('students')` setara dengan menulis `['students' => $students]`, sehingga di dalam view data dapat dipanggil menggunakan variabel `$students`.

3. Tambahkan kode di file `routes/web.php` seperti di bawah ini, lalu simpan.



```

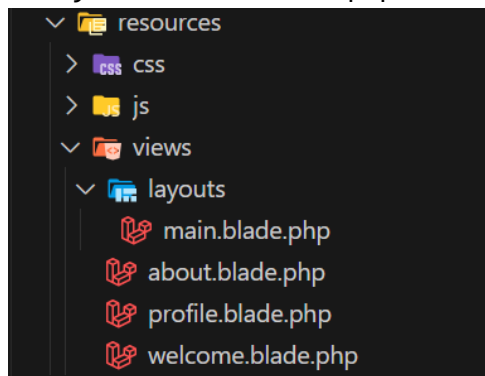
routes > web.php
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\StudentController;
5
6  Route::get('/students', [StudentController::class, 'index'])->name('students.index');
```

Penjelasan:

- `use App\Http\Controllers\StudentController;` → import StudentController agar bisa digunakan.
- `Route::get('/students', [...])` → membuat route GET /students.
- `[StudentController::class, 'index']` → ketika route /students diakses, Laravel akan menjalankan method index di StudentController.
- `->name('students.index')` → memberi nama route. Berguna untuk dipanggil di Blade dengan `route('students.index')`.

Berbeda dengan Modul 2 yang banyak menggunakan closure, pada modul ini route diarahkan ke method di dalam Controller. Cara ini lebih rapi karena logika aplikasi dipisahkan dari file `routes/web.php`. Named route `students.index` nantinya dipakai pada link navigasi di layout utama.

4. Lalu buat folder layouts dan buat file `main.blade.php` pada direktori tersebut: `resources/views/layouts/main.blade.php`



5. Lalu isi file dengan kode di bawah ini, lalu simpan.

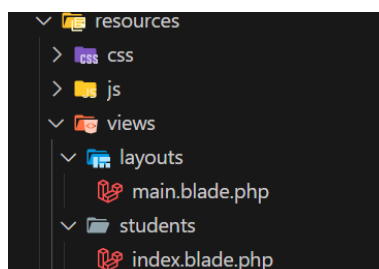
```
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4     <meta charset="UTF-8">
5     <title>@yield('title') | Praktikum Laravel</title>
6     <style>
7         body { font-family: Arial, sans-serif; margin: 20px; }
8         header, footer { background: #f5f5f5; padding: 10px; text-align: center; }
9         nav a { margin: 0 10px; text-decoration: none; }
10        .card { border: 1px solid #ddd; padding: 15px; margin: 10px 0; border-radius: 6px; }
11        .course { background: #eef; margin: 3px; padding: 5px; display: inline-block; border-radius: 4px; }
12    </style>
13 </head>
14 <body>
15     <header>
16         <h1>Praktikum Laravel</h1>
17         <nav>
18             <a href="{{ route('students.index') }}">Mahasiswa</a>
19         </nav>
20     </header>
21
22     <main>
23         @yield('content')
24     </main>
25
26     <footer>
27         <p>&copy; 2025 Praktikum Laravel - Nama Anda</p>
28     </footer>
29 </body>
30 </html>
31
```

Penjelasan:

- `@yield('title')` → placeholder judul halaman yang bisa diisi oleh view turunan.
- `style { ... }` → CSS sederhana agar tampilan lebih menarik.
- `<a href="{{ route('students.index') }}">` → link ke halaman mahasiswa dengan **named route**.
- `@yield('content')` → placeholder untuk isi konten halaman.

Layout utama ibarat sebuah "cetakan" halaman. Bagian yang selalu sama di setiap halaman (head, CSS, navigasi, dan footer) cukup ditulis sekali di file ini, sedangkan `@yield` menandai bagian kosong yang nantinya diisi oleh masing-masing halaman. Dengan begitu, jika ingin mengubah menu atau warna tampilan, Anda cukup mengubah satu file ini saja.

6. Lalu buat folder `students` dan buat file di direktori tersebut:
`resources/views/students/index.blade.php`



7. Isi file index.blade.php dengan kode di bawah ini, lalu simpan.

```
1  @extends('layouts.main')
2
3  @section('title', 'Daftar Mahasiswa')
4
5  @section('content')
6      <h2>Daftar Mahasiswa</h2>
7
8      @foreach($students as $student)
9          <div class="card">
10              <h3>{{ $student['name'] }}</h3>
11              <p><strong>Jurusan:</strong> {{ $student['major'] }}</p>
12              <p><strong>Umur:</strong> {{ $student['age'] }} tahun</p>
13
14              @if($student['age'] >= 22)
15                  <p>Status: <span style="color:green;">Senior</span></p>
16              @else
17                  <p>Status: <span style="color:blue;">Junior</span></p>
18              @endif
19
20              <h4>Mata Kuliah:</h4>
21              @foreach($student['courses'] as $course)
22                  <span class="course">{{ $course }}</span>
23              @empty
24                  <p>Tidak ada mata kuliah terdaftar.</p>
25              @endforeach
26          </div>
27      @endforeach
28  @endsection
29
```

Penjelasan:

- @extends('layouts.main') → mewarisi layout utama.
- @section('title', 'Daftar Mahasiswa') → mengisi placeholder @yield('title') di layout.
- @foreach(\$students as \$student) → looping semua data mahasiswa.
- {{ \$student['name'] }} → menampilkan nama mahasiswa.
- @if(\$student['age'] >= 22) → menampilkan status “Senior” jika umur ≥ 22, selain itu “Junior”.
- @foreach → looping daftar mata kuliah. Jika kosong, tampilkan pesan @empty.

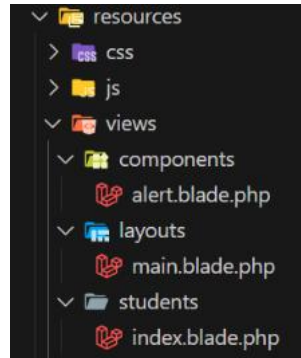
Catatan: @yield('nama') di layout diisi oleh @section('nama') di view turunan. Setiap directive wajib ditutup, misalnya @if dengan @endif dan @foreach dengan @endforeach.

3.4.3 Reusable Components

Komponen Blade adalah potongan tampilan yang dapat dipakai berulang kali di banyak halaman, misalnya kotak pesan (alert), tombol, atau kartu. Komponen disimpan di folder

resources/views/components dan dipanggil menggunakan tag berawalan x-, misalnya <x-alert>. Data dikirim ke komponen melalui atribut, seperti type dan message.

1. Selanjutnya kita akan menambahkan komponen blade. Buat folder pada views dan tambahkan file alert: resources/views/components/alert.blade.php



2. Lalu isi file tersebut dengan kode di bawah ini, lalu simpan

```
1 <div style="padding:10px; border:1px solid {{ $type === 'error' ? 'red' : 'green' }}; margin:10px 0;">
2   <strong>{{ ucfirst($type) }}:</strong> {{ $message }}
3 </div>
```

Penjelasan:

- {{ \$type === 'error' ? 'red' : 'green' }} → operator ternary untuk menentukan warna border.
- {{ ucfirst(\$type) }} → mengubah huruf pertama menjadi kapital (contoh: error → Error).
- {{ \$message }} → menampilkan pesan yang dikirim saat komponen dipanggil.

Catatan: file di folder resources/views/components otomatis dikenali sebagai komponen <x-alert>. Nilai \$type dan \$message diambil dari atribut saat komponen dipanggil.

3. Kembali ke file view resources/views/students/index.blade.php dan tambahkan kode alert seperti gambar di bawah ini, lalu simpan.

```

@extends('layouts.main')

@section('title', 'Daftar Mahasiswa')

@section('content')
    <h2>Daftar Mahasiswa</h2>

    {{-- Komponen Alert --}}
    <x-alert type="success" message="Data mahasiswa berhasil dimuat." />

    @foreach($students as $student)
        <div class="card">
            <h3>{{ $student['name'] }}</h3>
            <p><strong>Jurusan:</strong> {{ $student['major'] }}</p>
            <p><strong>Umur:</strong> {{ $student['age'] }} tahun</p>

            @if($student['age'] >= 22)
                <p>Status: <span style="color:green;">Senior</span></p>
            @else
                <p>Status: <span style="color:blue;">Junior</span></p>
            @endif

            <h4>Mata Kuliah:</h4>
            @forelse($student['courses'] as $course)
                <span class="course">{{ $course }}</span>
            @empty
                <p>Tidak ada mata kuliah terdaftar.</p>
            @endforelse
        </div>
    @endforeach
@endsection

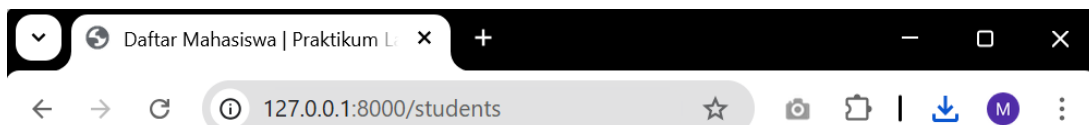
```

Penjelasan:

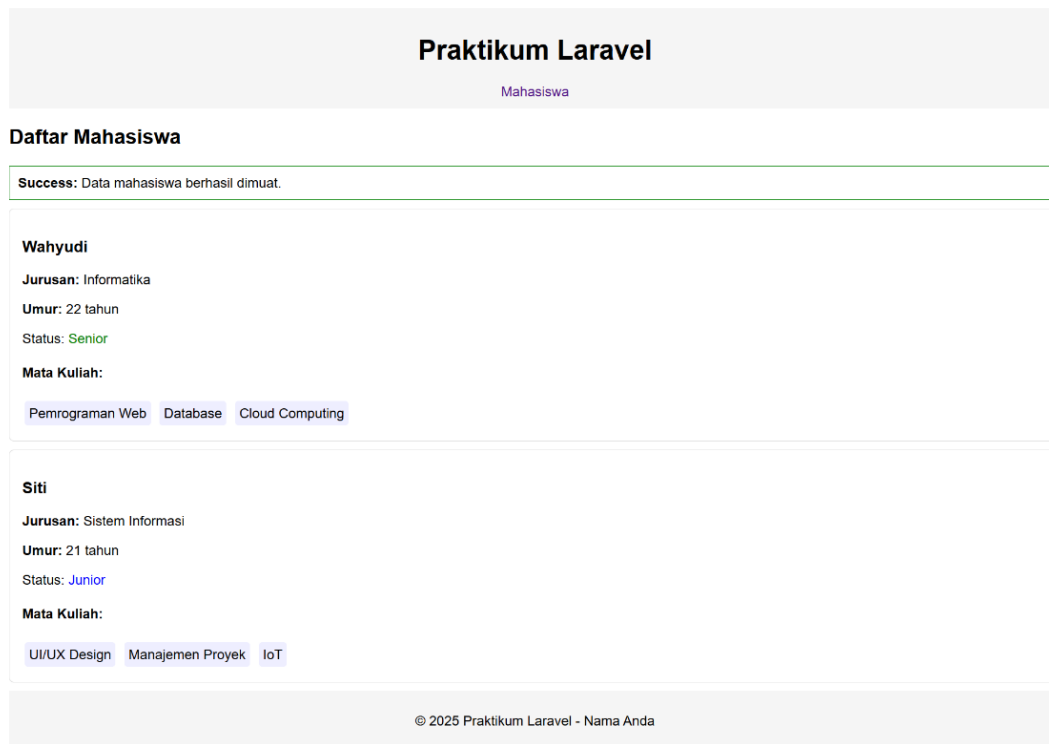
- `<x-alert type="success" message="..." />` → saat dipanggil akan berwarna hijau.
- `<x-alert type="error" message="..." />` → saat dipanggil akan berwarna merah.

Satu komponen cukup dibuat sekali, lalu dapat dipanggil berkali-kali dengan atribut yang berbeda. Inilah yang dimaksud reusable: jika desain alert ingin diubah, cukup ubah file `alert.blade.php` dan seluruh halaman yang memakainya akan ikut berubah.

4. Jalankan server melalui terminal (php artisan serve), lalu tambahkan `/students` pada URL.



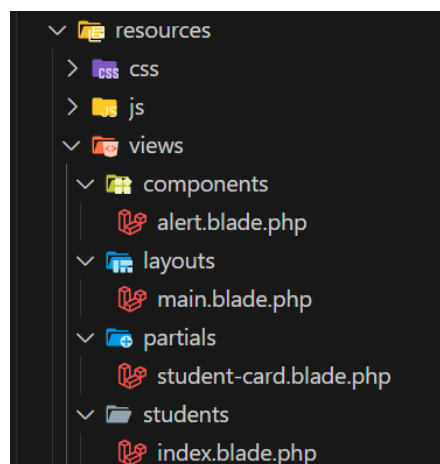
Maka hasil halaman akan tampil seperti berikut.



3.4.4 Partial Sederhana

Partial adalah potongan file Blade yang disisipkan ke dalam view lain menggunakan directive `@include`. Partial cocok untuk memecah view yang panjang menjadi bagian-bagian kecil, misalnya header, sidebar, atau kartu data, sehingga kode lebih mudah dibaca dan dirawat. Pada bagian ini, tampilan satu kartu mahasiswa akan dipisahkan ke file tersendiri.

1. Buat folder `partials` dan file `student-card`: `resources/views/partials/student-card.blade.php`



2. Tambahkan kode pada file tersebut seperti di bawah ini, lalu simpan

```
b.php  main.blade.php  index.blade.php  student-card.blade.php X  alert
resources > views > partials > student-card.blade.php
1  <div class="card">
2      <h3>{{ $student['name'] }}</h3>
3      <p><strong>Jurusan:</strong> {{ $student['major'] }}</p>
4      <p><strong>Umur:</strong> {{ $student['age'] }} tahun</p>
5
6      @if($student['age'] >= 22)
7          <p>Status: <span style="color:green;">Senior</span></p>
8      @else
9          <p>Status: <span style="color:blue;">Junior</span></p>
10     @endif
11
12     <h4>Mata Kuliah:</h4>
13     @forelse($student['courses'] as $course)
14         <span class="course">{{ $course }}</span>
15     @empty
16         <p>Tidak ada mata kuliah terdaftar.</p>
17     @endforelse
18 </div>
19
```

Penjelasan: Partial ini hanya berfokus pada tampilan satu mahasiswa. Variabel \$student yang digunakan di dalamnya dikirim dari view pemanggil melalui @include.

3. Buka file resources/views/students/index.blade.php lalu ubah kode seperti di bawah ini dan simpan. Jalankan kembali server, maka tampilan akan sama seperti sebelumnya.

```
b.php  main.blade.php  index.blade.php ●  student-card.blade.php  alert.blade.php
resources > views > students > index.blade.php
1  @extends('layouts.main')
2
3  @section('title', 'Daftar Mahasiswa')
4
5  @section('content')
6      <h2>Daftar Mahasiswa</h2>
7
8      {{-- Komponen Alert --}}
9      <x-alert type="success" message="Data mahasiswa berhasil dimuat." />
10
11     {{-- Looping dengan include --}}
12     @foreach($students as $student)
13         @include('partials.student-card', ['student' => $student])
14     @endforeach
15 @endsection
16
17
18
```

Penjelasan:

- `@include('partials.student-card', ['student' => $student])`
→ memanggil partial `student-card.blade.php` dan mengoper variabel `$student` ke dalamnya.
- Bedanya dengan `<x-alert>`: `@include` tidak punya konsep props/atribut seperti komponen, tapi langsung kirim variabel.
- Dengan cara ini, kode jadi lebih bersih karena view utama tidak terlalu panjang.

Kapan memakai komponen dan kapan memakai partial? Gunakan komponen (`<x-...>`) untuk elemen antarmuka kecil yang dipakai di banyak halaman dan menerima atribut, seperti alert atau tombol. Gunakan partial (`@include`) untuk memecah bagian halaman yang besar agar view utama lebih ringkas. Keduanya sama-sama bertujuan agar kode tidak ditulis berulang-ulang.

3.5 Tugas Praktikum

1. Badan Penanggulangan Bencana Daerah (BPBD) Kabupaten Bandung ingin membangun sistem pelaporan banjir sederhana bernama "LaporBanjir" agar warga dapat melaporkan kejadian banjir di wilayahnya dengan cepat. Sistem ini terdiri atas tiga halaman, yaitu halaman form pelaporan (berisi nama pelapor, lokasi kejadian, dan tinggi genangan air) yang dikirim melalui method POST, halaman konfirmasi yang menampilkan kembali data yang telah dikirim, serta halaman daftar laporan yang menampilkan beberapa contoh laporan banjir. Karena masih berupa prototipe, sistem dibuat menggunakan Laravel tanpa database, sehingga data laporan cukup ditulis dalam bentuk array pada Controller. BPBD juga menginginkan seluruh halaman memiliki tampilan yang seragam dan mudah dikembangkan. Oleh karena itu, rancang dan buatlah aplikasi tersebut menggunakan Blade Template dengan ketentuan sebagai berikut:
 - a. Layout utama: buat satu file `resources/views/layouts/app.blade.php` yang berisi header nama sistem, menu navigasi (menggunakan named route), footer, dan CSS. Setiap halaman wajib mewarisi layout ini menggunakan `@extends`, `@section`, dan `@yield`.
 - b. Halaman daftar laporan: tampilkan data laporan menggunakan `@foreach` atau `@forelse`. Tentukan status genangan menggunakan `@if/@elseif/@else`, yaitu kurang dari 30 cm "Waspada", 30–70 cm "Siaga", dan lebih dari 70 cm "Awat".
 - c. Halaman konfirmasi: tampilkan pesan bahwa laporan berhasil dikirim menggunakan komponen Blade `<x-alert>`.
 - d. Kartu laporan: tampilkan setiap laporan melalui partial `resources/views/partials/laporan-card.blade.php` yang dipanggil dengan `@include`.
 - e. Route: pastikan setiap halaman (form, konfirmasi, dan daftar laporan) dapat diakses melalui route.
2. Terapkan Blade Template pada rancangan Tugas Akhir Pemrograman Web Semester 2 Saudara dengan mengubah seluruh tampilan (view) menjadi Blade View pada Laravel, dengan ketentuan sebagai berikut:

- a. Layout utama: buat satu layout utama yang memuat bagian yang sama di setiap halaman (header, menu navigasi, footer, dan CSS), lalu wariskan ke seluruh halaman menggunakan @extends, @section, dan @yield.
- b. Directive Blade: gunakan minimal satu directive percabangan (@if) dan satu directive perulangan (@foreach atau @forelse) untuk menampilkan data pada halaman yang sesuai.
- c. Komponen/partial: manfaatkan minimal satu komponen Blade (<x-...>) atau partial (@include) untuk bagian tampilan yang digunakan berulang.
- d. Route: pastikan setiap halaman dapat diakses melalui route dan tautan navigasi menggunakan named route.

3.6 Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

1. Halaman Sampul (Cover)

Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.

2. Tujuan Praktikum

Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.

3. Alat dan Bahan

Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya XAMPP/Laragon, Composer, PHP, Laravel Framework, Visual Studio Code, dan Git.

4. Langkah-Langkah Praktikum

Menguraikan tahapan pengerjaan secara runtut, mulai dari pembuatan route dan controller, penyusunan layout utama (@extends, @section, @yield), penggunaan directive Blade (@if, @foreach, @forelse), pembuatan komponen (<x-...>) dan partial (@include), hingga pengujian aplikasi melalui browser. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.

5. Hasil dan Dokumentasi Program

Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya halaman form pelaporan, halaman konfirmasi (berserta tampilan komponen alert), dan halaman daftar laporan dengan status genangan, serta halaman-halaman Tugas Akhir yang telah diubah menjadi Blade View. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat.

6. Analisis dan Pembahasan

Membahas alasan pembagian layout, komponen, dan partial pada aplikasi, perbedaan tampilan sebelum dan sesudah menggunakan Blade, serta kendala atau galat yang ditemukan beserta cara penyelesaiannya.

7. Kesimpulan

Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.

8. Lampiran berisi Link Repositori GitHub

Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu dan memuat kode program lengkap.

9. Presentasi Hasil Praktikum

Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.

TERIMAKASIH